PocketSmart AI

Your Smart Budget & Recommendation Assistant

Project Report:

Project Title: PocketSmart AI – Your Smart Budget & Recommendation Assistant

Technology: FastAPI, Gemini 1.5 Flash Pro, HTML, CSS, JavaScript, Jinja2
Application Type: Generative AI-Based Budget Planning and Recommendation System

1.Introduction:

In today's digital environment, users have access to a large number of online shopping and service platforms. Although this provides more choices, it also makes decision-making more difficult.

For example, a person decorating a house may need to search for furniture, lighting, fans, dining tables, and decorative items across multiple platforms. Similarly, someone planning a party needs to consider food, venue, decorations, entertainment, and accommodation.

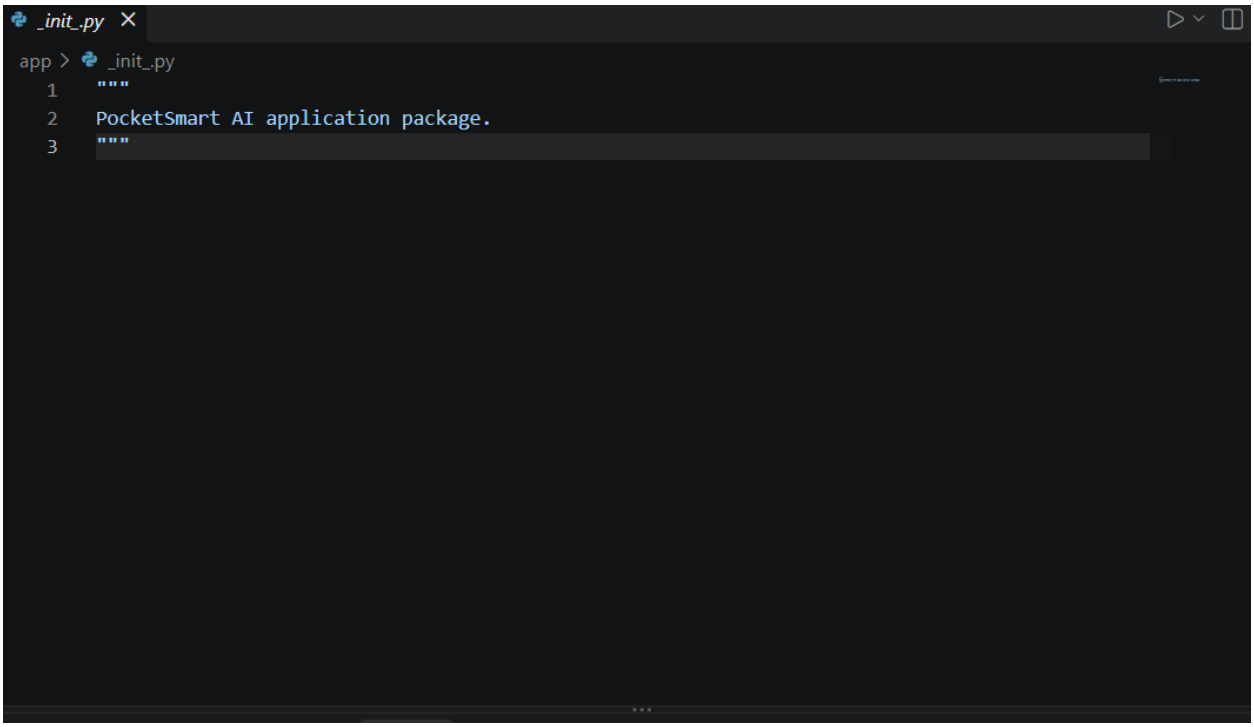
Traditional search-based approaches require users to manually compare products and prices.

PocketSmart AI provides an intelligent alternative.

The system accepts a user's:

- Budget
- Preferences
- Requirements
- Occasion
- Number of guests
- Room details
- Style preferences
- Optional outfit images

The AI then processes this information and generates relevant recommendations.

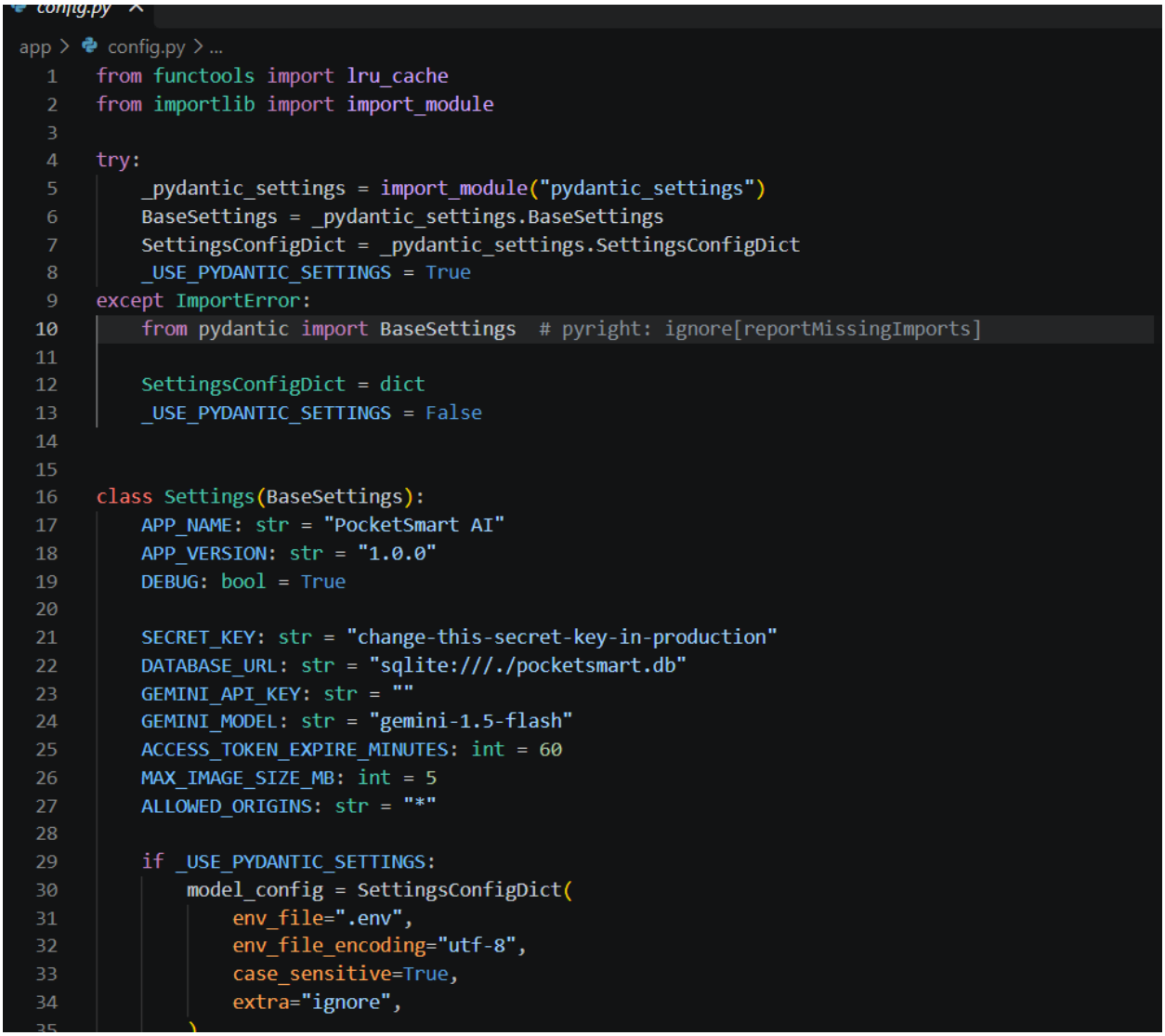


2.Problem Statement:

Users often face the following challenges when planning purchases or events:

- Difficulty maintaining a fixed budget
- Too many product and service choices
- Time-consuming manual comparison
- Lack of personalized recommendations
- Difficulty finding products matching a specific occasion
- Difficulty coordinating products with personal preferences
- Lack of a unified planning system across different categories

PocketSmart AI aims to address these challenges through an intelligent, budget-aware recommendation system.



```python
from functools import lru_cache
from importlib import import_module

try:
    _pydantic_settings = import_module("pydantic_settings")
    BaseSettings = _pydantic_settings.BaseSettings
    SettingsConfigDict = _pydantic_settings.SettingsConfigDict
    _USE_PYDANTIC_SETTINGS = True
except ImportError:
    from pydantic import BaseSettings  # pyright: ignore[reportMissingImports]

    SettingsConfigDict = dict
    _USE_PYDANTIC_SETTINGS = False


class Settings(BaseSettings):
    APP_NAME: str = "PocketSmart AI"
    APP_VERSION: str = "1.0.0"
    DEBUG: bool = True

    SECRET_KEY: str = "change-this-secret-key-in-production"
    DATABASE_URL: str = "sqlite:///./pocketsmart.db"
    GEMINI_API_KEY: str = ""
    GEMINI_MODEL: str = "gemini-1.5-flash"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
    MAX_IMAGE_SIZE_MB: int = 5
    ALLOWED_ORIGINS: str = "*"

    if _USE_PYDANTIC_SETTINGS:
        model_config = SettingsConfigDict(
            env_file=".env",
            env_file_encoding="utf-8",
            case_sensitive=True,
            extra="ignore",
        )
```

3.Proposed Solution:

PocketSmart AI provides a centralized platform where users can enter their requirements and receive AI-generated recommendations.

The system follows this general process:

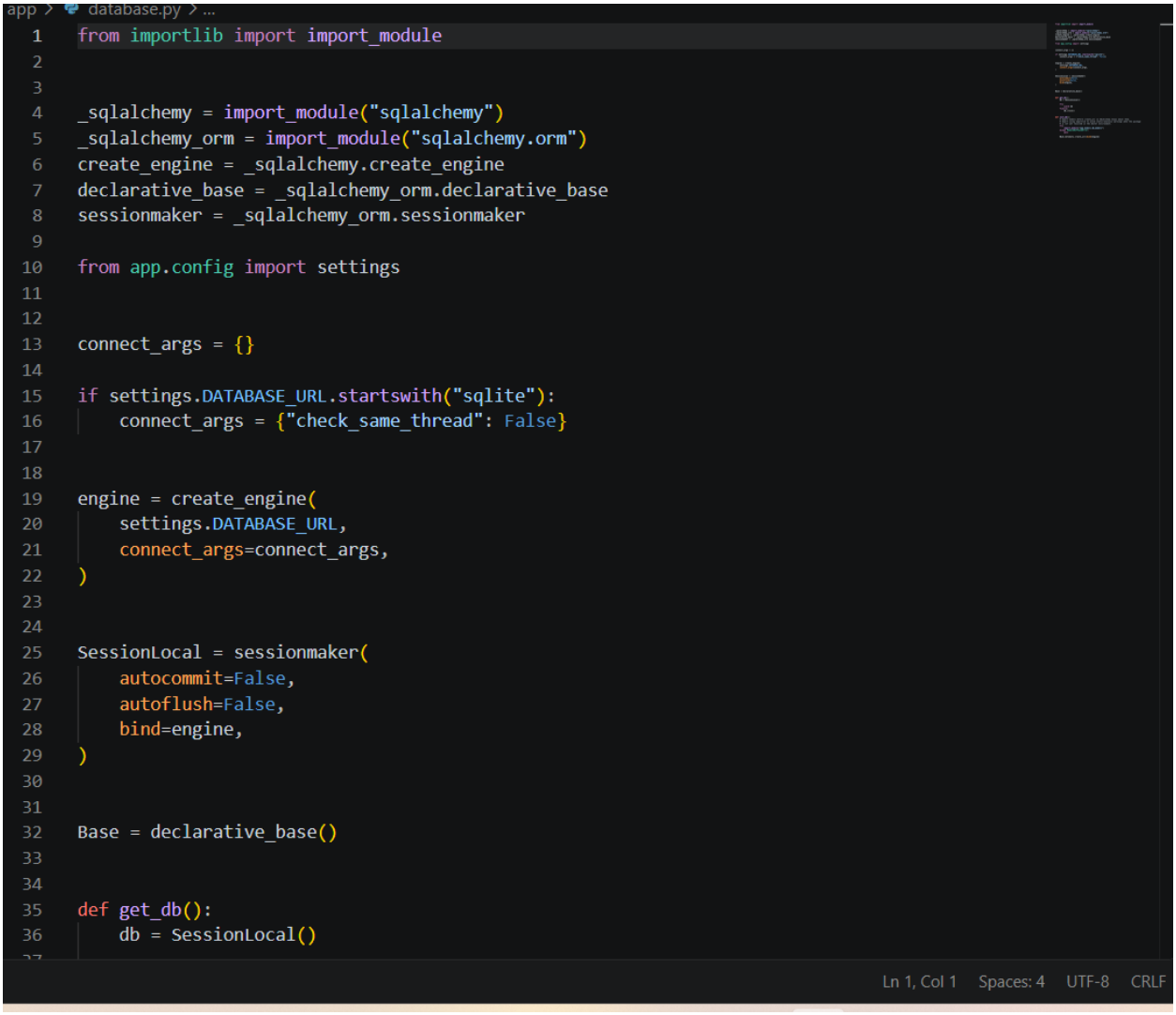
User Input → Budget Analysis → Context Understanding → AI Processing → Recommendation Generation → Results Display

The platform uses Gemini 1.5 Flash Pro to process text, numerical information, and optional image inputs.

The recommendations are designed around three important factors:

- **Budget**
- **User preferences**
- **Context**

This allows the system to provide recommendations that are more relevant to the user's specific requirements.

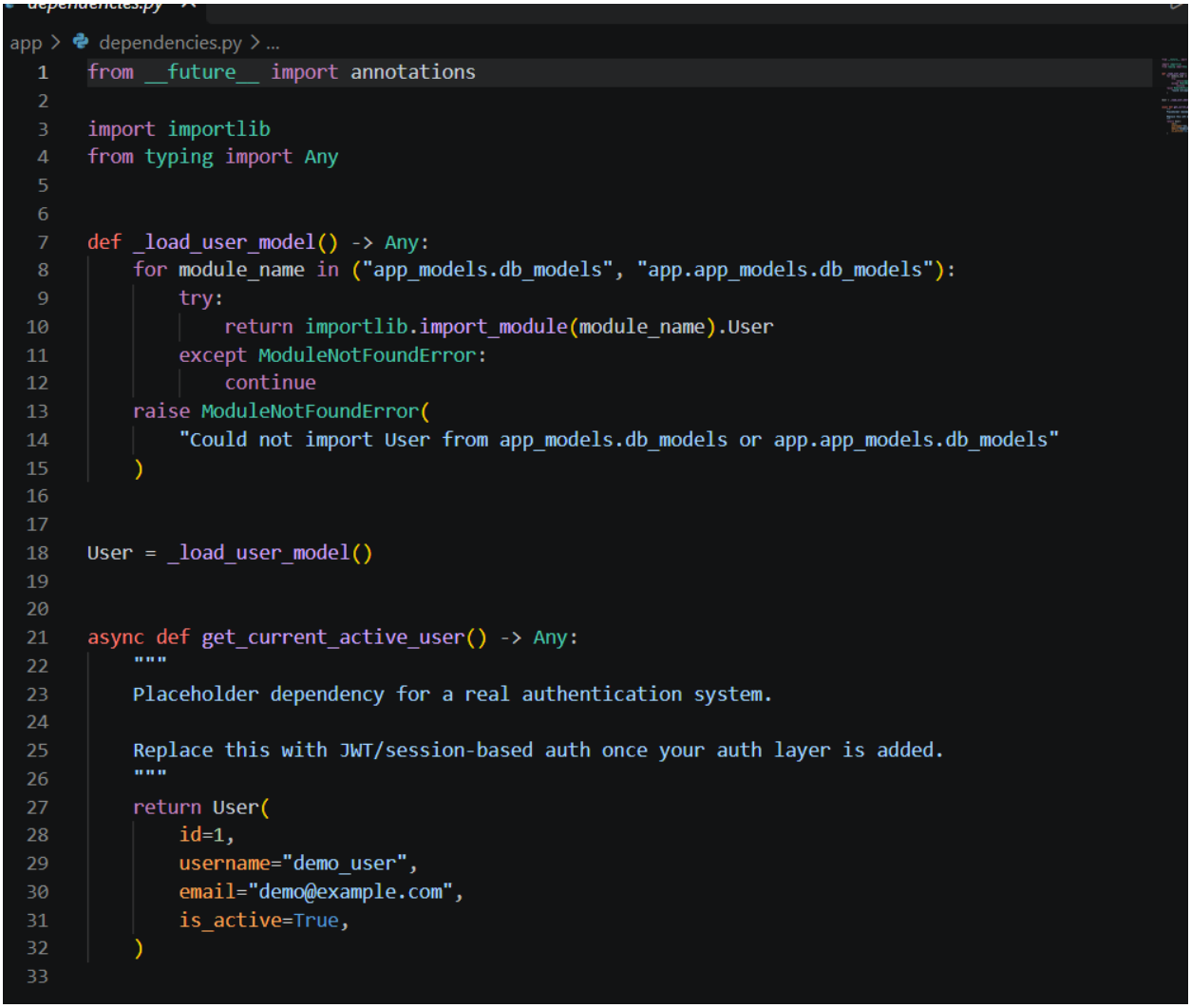


4.Objectives:

The main objectives of PocketSmart AI are:

1. To develop an AI-powered budget planning assistant.
2. To generate personalized recommendations.
3. To help users manage budgets efficiently.

4. To provide recommendations across multiple lifestyle categories.
5. To integrate Generative AI into practical everyday planning.
6. To support optional image-based analysis for jewelry recommendations.
7. To provide a simple and interactive user interface.
8. To maintain user sessions and recommendation history.
9. To provide platform-aware recommendations.
10. To reduce the time required for manual product and service comparison.



5.Major Features:

1. User Registration and Login:

Users can create accounts and securely log into the application.

The system provides:

- Registration
- Login
- Logout
- Authentication
- Session management
- User-specific recommendation history



2. Home Interior Planner:

The Home Interior Planner allows users to enter their home-related requirements.

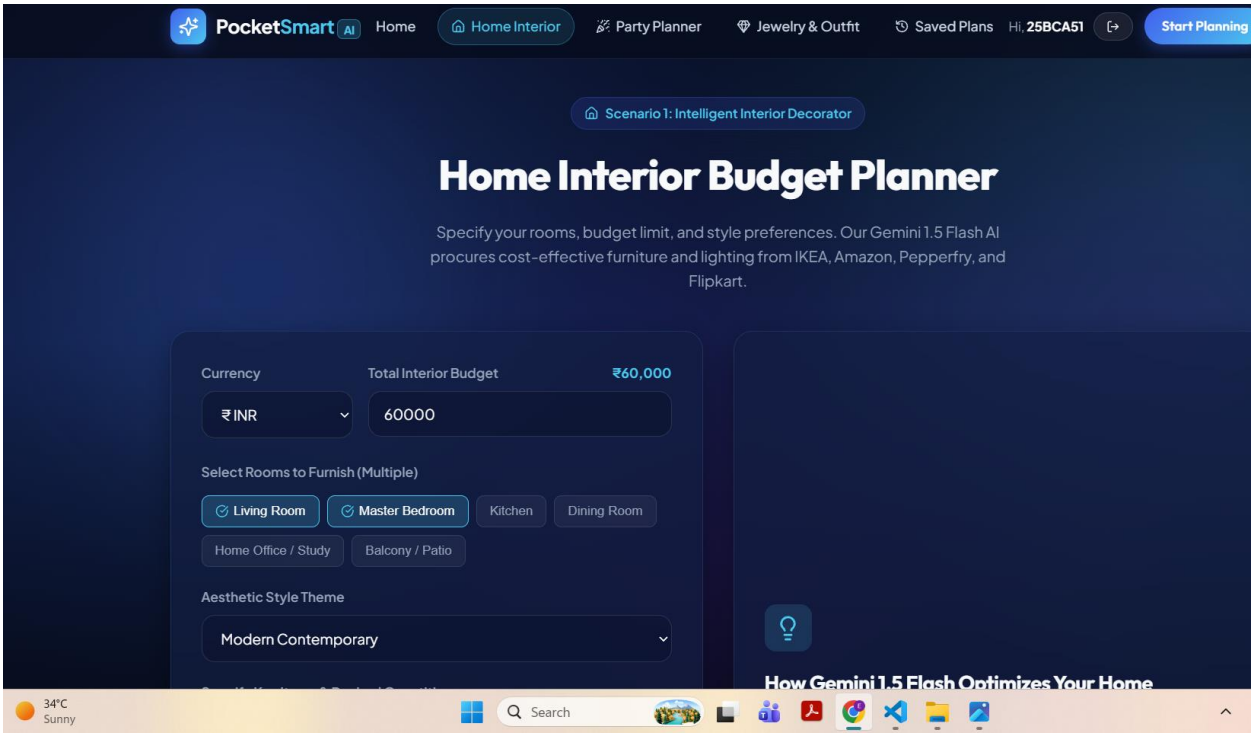
Users can specify:

- Total budget
- Room type
- Required items
- Quantity
- Style preferences

The system generates recommendations for items such as:

- Furniture
- Lighting
- Ceiling fans
- Dining tables
- Decorative products
- Other interior items

Recommendations can be associated with platforms such as Amazon and IKEA.



3.Party Budget Planner:

The Party Planner helps users organize an event within a specified budget.

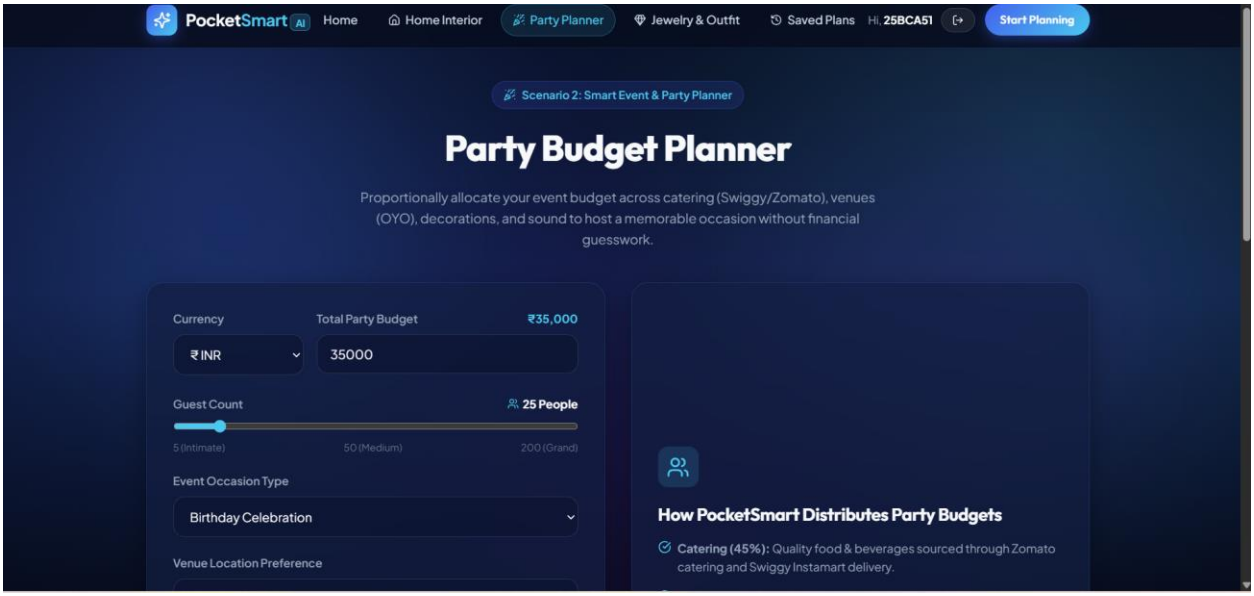
Users provide information such as:

- Total budget
- Number of guests
- Event type
- Venue details
- Food requirements
- Decoration requirements

The AI can divide the available budget across areas such as:

- Catering
- Decorations
- Entertainment
- Venue
- Accommodation

The project documentation identifies platforms such as Swiggy, Zomato, and OYO as relevant sources for party-related recommendations.



4. Jewelry Budget Planner:

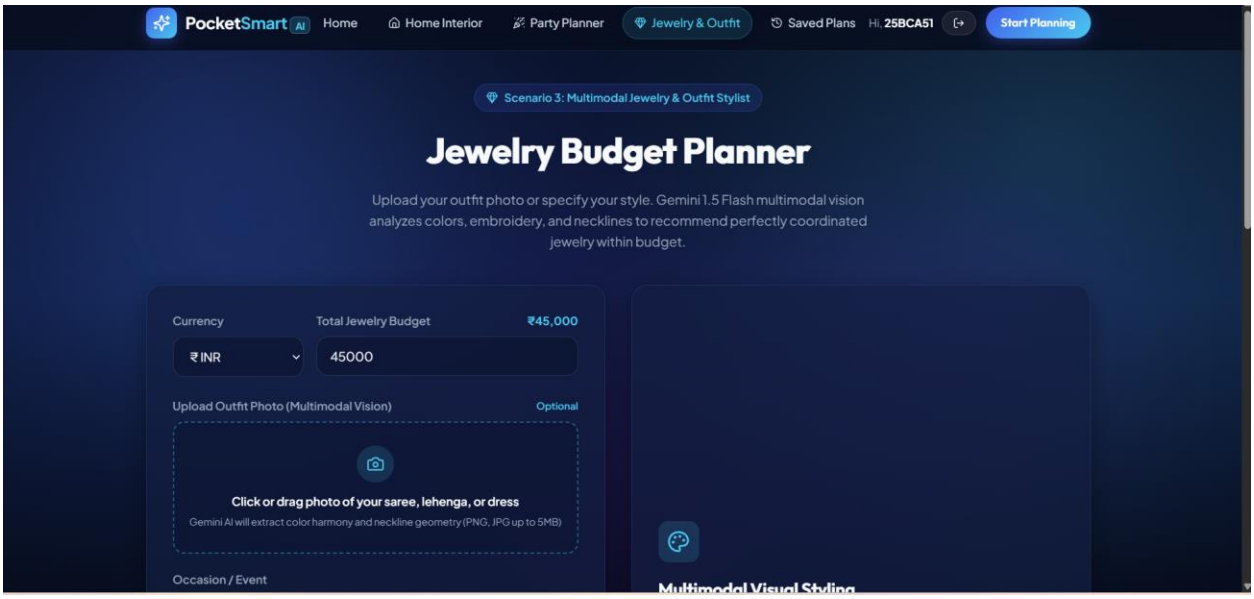
The Jewelry Planner helps users find jewelry according to their:

- Budget
- Occasion
- Style
- Outfit
- Color preferences

Users can optionally upload an outfit image.

Gemini's multimodal capabilities are used to analyze the available information and generate suitable jewelry recommendations.

Potential recommendation platforms include Amazon and Flipkart.



6.System Architecture:

Architecture Flow

User

↓

Frontend – HTML / CSS / JavaScript / Jinja2

↓

FastAPI Backend

↓

Authentication & Session Management

↓

Recommendation Services

↓

Gemini 1.5 Flash Pro

↓

Product / Service Sources

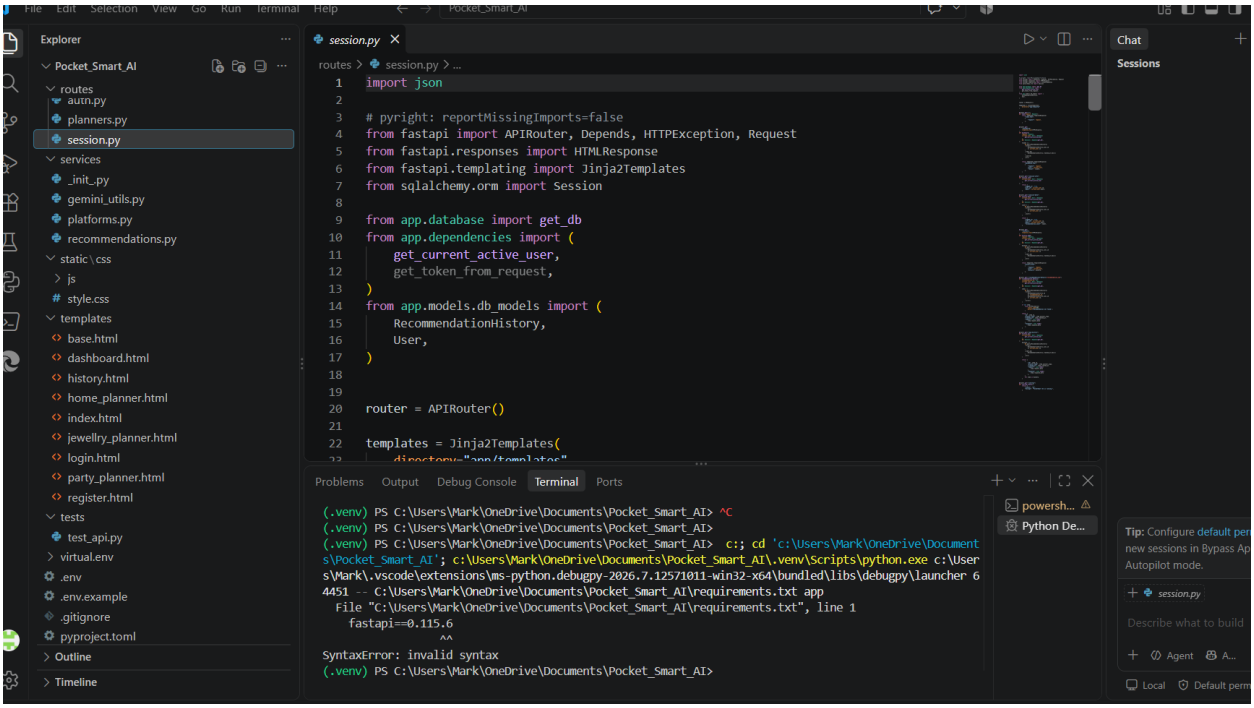
↓

AI-Generated Recommendations

↓

Frontend Results Page

The documentation describes the system as a modular platform combining a FastAPI backend, Gemini-based AI layer, third-party product/service sources, and an interactive frontend.



7. Backend Architecture:

The backend is organized into modular components.

Main Components:

Routes

Responsible for handling application endpoints.

Examples include:

- `/generate-home`

- `/generate-party`
- `/generate-jewelry`
- `/register`
- `/login`
- `/logout`
- `/token`
- `/session-info`
- `/session-data`
- `/history`

Services

Responsible for:

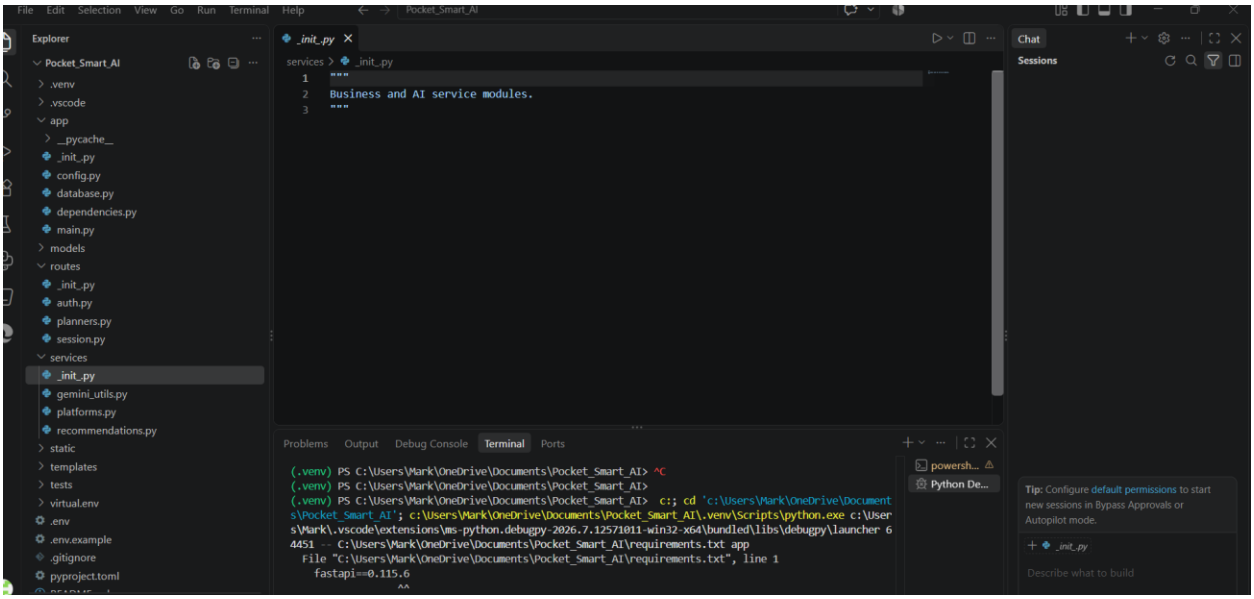
- Recommendation logic
- Gemini API communication
- Prompt generation
- Budget processing
- AI response handling

Models

Responsible for defining:

- Input structures
- Output structures
- User information
- Recommendation data

The project documentation specifically describes modular organization around routes, services, and models.



8. AI Integration:

Gemini 1.5 Flash Pro forms the central AI component of PocketSmart AI.

The AI layer processes:

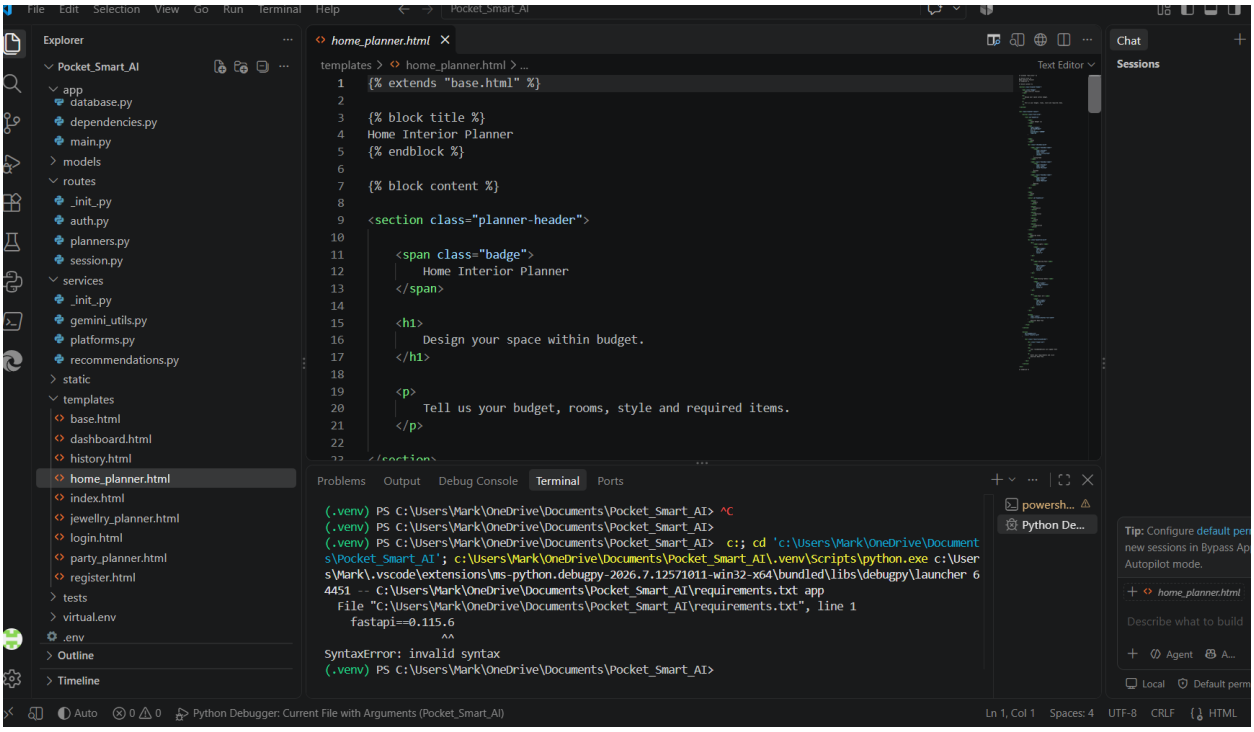
- User text
- Budget values
- Preferences
- Planning requirements
- Contextual information
- Optional images

The system uses AI prompts designed specifically for each planner.

Home Planner AI

Analyzes:

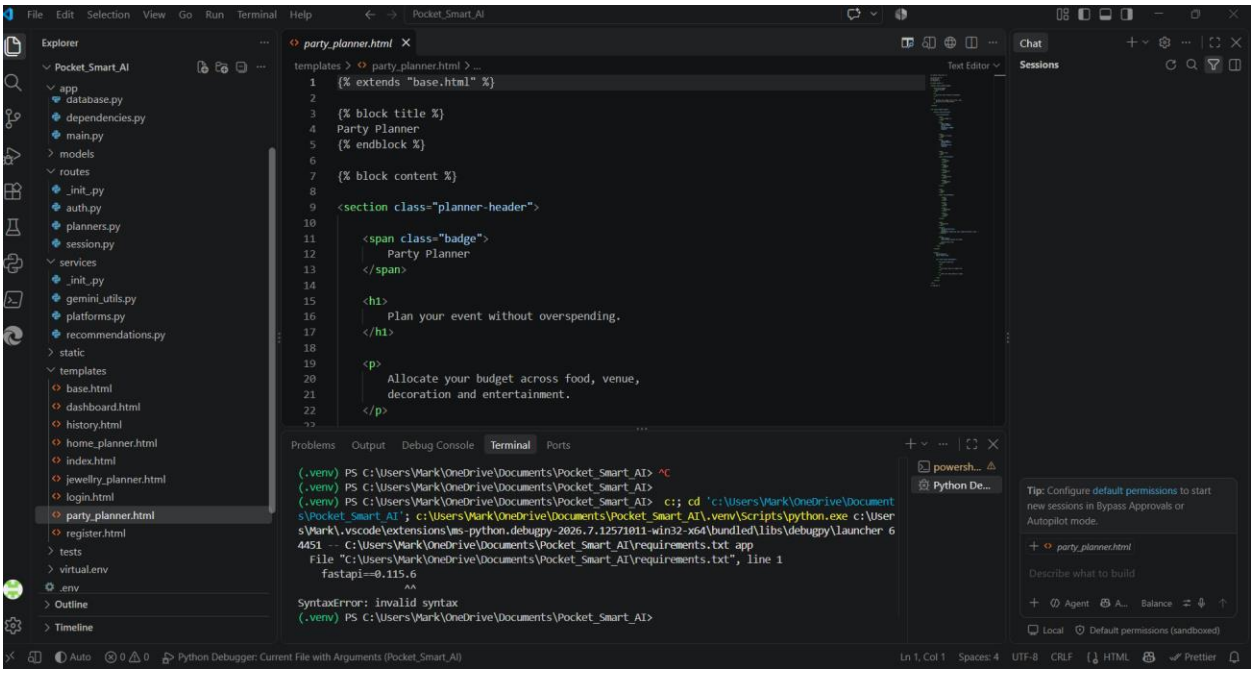
- Room requirements
- Item quantities
- Budget
- Style



Party Planner AI

Analyzes:

- Event type
- Guest count
- Budget
- Venue requirements
- Food and decoration requirements

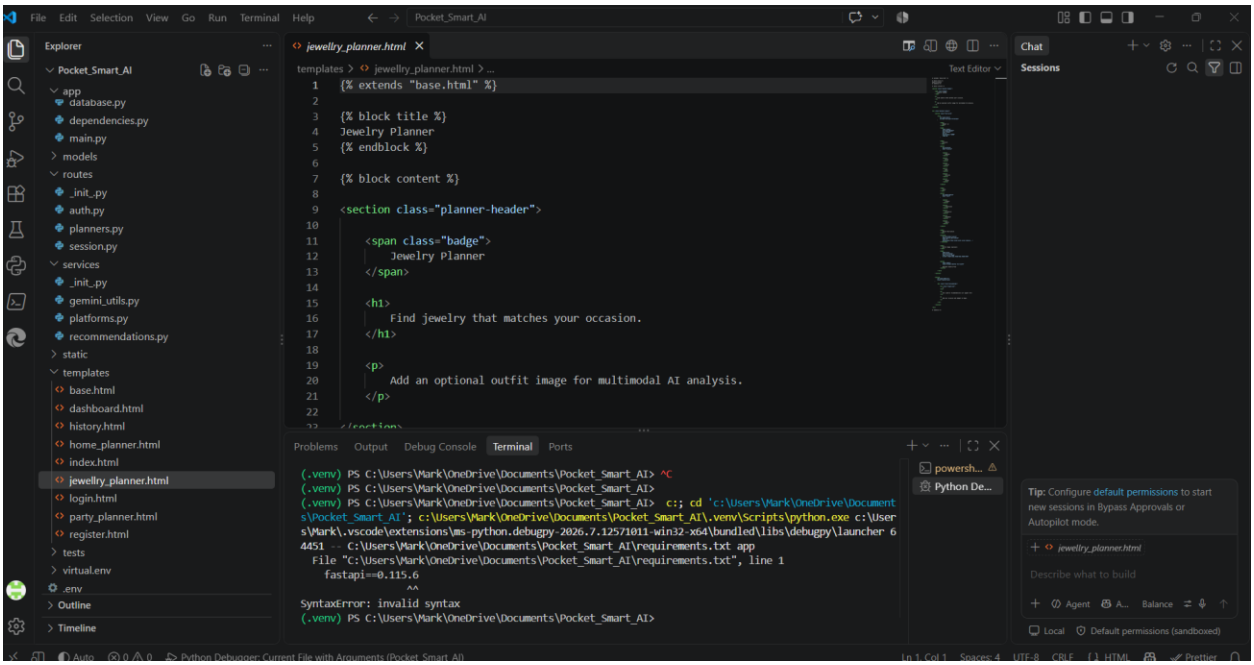


Jewelry Planner AI

Analyzes:

- Occasion
- Budget
- Style
- Outfit information
- Optional outfit image

The documentation identifies multimodal processing as an important capability, particularly for the jewelry planner.



9. Expected Outcome

The expected outcome of PocketSmart AI is a practical AI-powered assistant capable of helping users plan lifestyle-related purchases and events according to their available budgets.

The application combines:

Artificial Intelligence + Budget Planning + Personalization + Recommendations + Web Technology

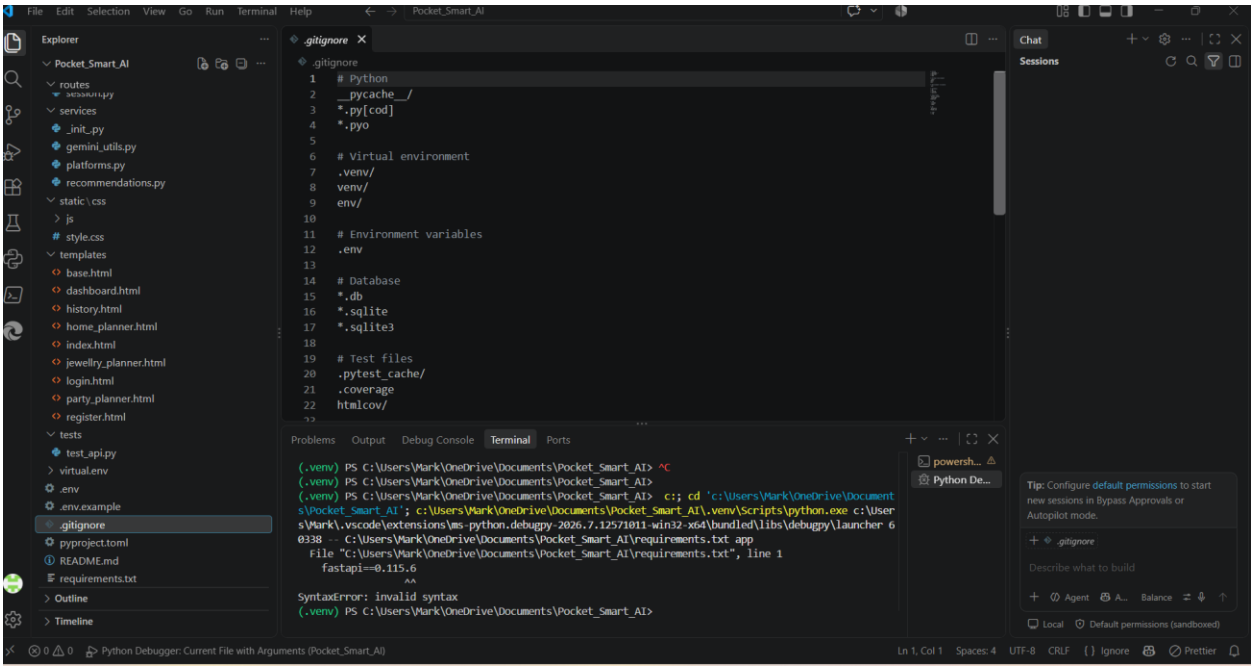
to create an interactive planning experience.

17. Advantages

PocketSmart AI provides several useful capabilities:

- AI-powered recommendations
- Budget-aware planning
- Multiple planning categories
- Personalized suggestions
- Multimodal image support
- Cross-platform recommendation concept

- User authentication
- Recommendation history
- Modular backend architecture
- Responsive web interface



10. Conclusion

PocketSmart AI demonstrates how Generative AI can be applied to everyday budget planning and recommendation tasks.

The system combines FastAPI, Gemini 1.5 Flash Pro, HTML, CSS, JavaScript, Jinja2, authentication, and recommendation services to create an intelligent platform for Home Interior, Party, and Jewelry planning.

By accepting budgets, preferences, contextual requirements, and optional images, the application is designed to provide personalized recommendations while helping users remain within their defined financial limits.

The project therefore presents a practical implementation of Generative AI for personalized lifestyle planning and recommendations.

Team Members:

PocketSmart AI

Your Smart Budget & Recommendation Assistant

Team Members

1. Muskan Tabassum – Team Leader
2. Mubashira Maryam
3. Lakshanika
4. Nagajothi
5. Nakshatra